

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN

PRÁCTICA DE LABORATORIO N° 2

Implementacion del algoritmo quicksort y mersort

Autores:

Genesis V. Cabello M.

C.I:30.367.386

Maivet S. Pereira M.

C.I: 30.814.708

Profesor:

Jose Canache

Asignatura:

Arquitectura del Computador

Objetivo:

Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención a la gestión de registros, llamadas a funciones, estructuras de control, camino de datos, manejo de la pila y configuración/utilización del ambiente de trabajo.

1. Investigación Preliminar

1.1. ¿Qué diferencias existen entre registros temporales (\$t0-\$t9) y registros guardados (\$s0-\$s7) y cómo se aplicó esta distinción en la práctica?

La diferencia principal radica en la convención de llamadas a subrutinas. Los registros temporales (\$t0-\$t9) no se preservan a través de las llamadas a funciones; si una subrutina los modifica, el programa principal no espera recuperar su valor original. Por el contrario, los registros guardados (\$s0-\$s7) deben preservarse; si una subrutina los utiliza, debe guardar su valor en la pila al inicio y restaurarlo antes de retornar. En la práctica, los \$t se usaron para cálculos efímeros, variables de intercambio y contadores de bucles cortos. La dirección base se mantuvo en el registro de argumento \$a0, mientras que en Quicksort se utilizó temporalmente el registro \$s0 para almacenar el valor del pivote durante la partición”

1.2. ¿Qué diferencias existen entre los registros \$a0-\$a3,\$v0-\$v1,\$ra y cómo se aplicó esta distinción en la práctica?

\$a0-\$a3 (Argumentos): Se utilizan para pasar parámetros a las funciones. En la práctica, se emplearon para enviar la dirección del arreglo y los índices low y high a las subrutinas de ordenamiento.

\$v0-\$v1 (Valores de retorno): Se usan para devolver resultados de funciones y para configurar los códigos de las llamadas al sistema (syscalls). Se aplicaron para retornar pivotes en Quicksort y para imprimir valores o finalizar el programa.

\$ra (Return Address): Almacena la dirección de retorno a la que debe volver el programa tras ejecutar un salto y enlace (jal). Fue críticamente aplicado en la recursividad, exigiéndose guardar \$ra en la pila (\$sp) antes de cada llamada recursiva para evitar perder el punto de retorno.

1.3. ¿Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

Los accesos a memoria mediante instrucciones de carga (lw) y almacenamiento (sw) consumen significativamente más ciclos de reloj que las operaciones que ocurren exclusivamente dentro del banco de registros del procesador. Minimizar las lecturas/escrituras

en la memoria RAM y mantener las variables activas (como índices, pivotes y valores temporales de intercambio) dentro de los registros aceleró dramáticamente el tiempo de ejecución de ambos algoritmos.

1.4. ¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Las instrucciones de salto (beq, bne, j) generan riesgos de control en la segmentación del procesador, ya que pueden requerir que el pipeline se vacíe si la predicción de salto falla, desperdiciando ciclos de reloj. Minimizar y estructurar eficientemente los bucles de partición y mezcla permitió mantener un flujo de ejecución más lineal, mejorando el rendimiento del algoritmo.

1.5. ¿Cuáles son las diferencias de complejidad computacional entre el algoritmo Quicksort y el algoritmo Mergesort? ¿Qué implicaciones tiene esto para la implementación en un entorno MIPS32?

Quicksort tiene una complejidad promedio de $O(n \log n)$ pero un peor caso de $O(n^2)$. Su gran ventaja es que ordena in-place, lo que en MIPS se traduce en menos accesos a memoria y alta eficiencia, aunque su naturaleza recursiva exige un manejo intensivo de la pila. Mergesort garantiza $O(n \log n)$ en todos los casos, pero requiere un arreglo auxiliar temporal. En MIPS32, este uso de memoria extra implica un aumento drástico en las instrucciones lw y sw para mover datos entre el arreglo original y el auxiliar, ralentizando la ejecución en comparación con un Quicksort bien optimizado.

1.6. ¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

Instruction Fetch (IF): Se busca y lee la instrucción en la memoria de instrucciones usando el Program Counter (PC).

Instruction Decode (ID): Se decodifica la instrucción y se leen los operandos desde el banco de registros.

Execution (EX): La Unidad Aritmético Lógica (ALU) realiza la operación matemática, lógica, o calcula direcciones de memoria.

Memory Access (MEM): Si es necesario, se accede a la memoria de datos (para leer o escribir).

Write Back (WB): El resultado de la ALU o el dato leído de memoria se escribe de vuelta en el registro destino.

1.7. ¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

Predominaron las instrucciones tipo I y R. Las tipo I (lw, sw, addi, beq) fueron fundamentales por la necesidad constante de acceder a las posiciones de memoria de los arreglos mediante base + desplazamiento y para evaluar las condiciones de los bucles. Las tipo R (add, sub, slt) se usaron masivamente para la aritmética de punteros, incrementos de índices y comparaciones de valores necesarias durante el ordenamiento.

1.8. ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (j, beq, bne) en lugar de usar estructuras lineales?

El abuso de saltos destruye la localidad espacial y reduce drásticamente el rendimiento debido a los atascos en el pipeline (pipeline stalls). Cada salto tomado obliga a la arquitectura a descartar las instrucciones que ya habían empezado a cargarse, introduciendo latencia y aumentando el número de ciclos por instrucción (CPI) efectivos.

1.9. ¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

El modelo RISC ofrece instrucciones de longitud fija (32 bits) y un repertorio reducido, lo que permite una decodificación extremadamente rápida. Su amplio banco de 32 registros de propósito general es ideal para algoritmos de ordenamiento, ya que permite mantener múltiples variables locales, contadores y límites directamente en el procesador, minimizando el cuello de botella que representa el acceso a la memoria RAM.

1.10. ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

El modo "Step" permitió ejecutar el código instrucción por instrucción. Esto fue crucial para verificar el flujo de control, observar cómo se actualizaba el Program Counter (PC), confirmar que los saltos condicionales evaluaban las comparaciones correctamente y asegurar que el puntero de pila (\$sp) creciera y decreciera adecuadamente durante la profundidad de las llamadas recursivas.

1.11. ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

La ventana del Coprocesador 0 / Registros (Registers pane) fue la más útil para observar los cambios en tiempo real de las variables (índices, pivotes) y validar el estado del \$sp y \$ra. Conjuntamente, el visor del Data Segment (Segmento de datos) resultó indispensable para inspeccionar la memoria y corroborar visualmente cómo el arreglo intercambiaba sus posiciones y quedaba ordenado.

1.12. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R? (por ejemplo: add)

En una instrucción tipo R (add \$t0, \$t1, \$t2), el camino de datos se visualiza leyendo la instrucción (IF), enviando los registros origen (\$t1 y \$t2) a la ALU durante la decodificación (ID). En la fase EX, la ALU suma ambos valores. La fase MEM se ignora (no hay acceso a memoria de datos) y un multiplexor dirige el resultado de la ALU de vuelta al banco de registros para escribirse en \$t0 (WB).

1.13. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: lw)

Para una instrucción lw \$t0, 4(\$t1), el camino de datos implica buscar la instrucción (IF), leer el registro base \$t1 y pasar los 16 bits del inmediato ("4") por el extensor de signo (ID). La ALU suma la dirección base más el inmediato (EX). El resultado se usa como dirección para leer en la Memoria de Datos (MEM). Finalmente, el dato extraído de la memoria viaja por el bus para escribirse en el registro destino \$t0 (WB).

1.14. Justificar la elección del algoritmo alternativo

La implementación de Mergesort como alternativa a Quicksort se justifica cuando es estrictamente necesario garantizar un tiempo de ejecución estable de $O(n \log n)$ independientemente del grado de desorden inicial del arreglo, y cuando se requiere un método de ordenamiento estable (que preserve el orden relativo de elementos iguales). Sin embargo, a nivel de arquitectura MIPS, esta elección sacrifica memoria y ciclos de reloj debido al elevado costo de las lecturas/escrituras (lw/sw) necesarias para gestionar el arreglo auxiliar, haciendo que Quicksort (que opera in-place) sea superior en la mayoría de los escenarios de bajo nivel.

1.15. Análisis y Discusión de los Resultados

La práctica demostró exitosamente que programar en lenguaje ensamblador exige una comprensión profunda no solo de la lógica del algoritmo, sino del hardware subyacente. Los resultados de implementar Quicksort y Mergesort evidencian que el rendimiento en MIPS32 está altamente condicionado por la cantidad de accesos a memoria y el manejo del control de flujo. La recursividad demostró ser un desafío técnico que resalta la importancia de seguir rigurosamente las convenciones de registros y la manipulación de la pila (\$sp, \$ra, \$s0-\$s7). MARS probó ser un entorno excelente para conectar los conceptos teóricos del camino de datos y segmentación con la ejecución práctica de software.